TradeFolio Build Plan

Phase-by-phase plan for building TradeFolio by hand, following the project specification. Last updated 19 October 2026.

How to use this plan

- Work one task at a time, in order. After each task: run it, check the "Done when" line, then commit with a clear message.
- "Spec" points to numbered sections and FR requirements in the TradeFolio specification.
- **[YOU]** marks work where you write the scenario or calculate the expected numbers yourself, so the test is real proof and not just the code agreeing with itself.
- Keep docs/progress.md and docs/requirements-checklist.md updated at the end of every task.
- Do not skip ahead. Phase 4 (the concurrency proof) is the heart of the project, and everything after it depends on the trade path being correct.

Setup and conventions

- Repo folder: Portfolio-Trade-Notification. Solution name: TradeFolio. Windows with PowerShell and Docker Desktop. .NET SDK 10.0.402 pinned in global.json.
- Configuration comes from environment variables and a local .env file. .env is never committed. .env.example (placeholders only) is committed.
- Money, price and quantity use decimal in C# and numeric in PostgreSQL. Never float or double.
- Keep exactly one solution file (.sln or .slnx) in the repo root.
- Lesson learned: an empty (0-byte) global.json, Directory.Build.props or Directory.Packages.props makes every dotnet command crash with messages like "Root element is missing" or "input does not contain any JSON tokens". If you see those, check file sizes first.
- Empty folders are not tracked by git. That is fine; they fill up as you build.

Progress at a glance

- Phase 1: in progress. Tasks 1.1 and 1.2 are done. Task 1.3 is running (Swagger, health and environment config verified); finish the commit, then start 1.4.
- Phases 2 to 12: not started.

Phase 1: Foundation (week 1)

1.1 Solution skeleton (done)

- Build: global.json pinned to the SDK, the solution file, Directory.Build.props (nullable, implicit usings, warnings as errors), Directory.Packages.props (central package versions), and three projects: TradeFolio.Contracts, TradeFolio.Common and TradeService, with TradeService referencing the other two.

- Spec: sections 24, 25, 30.
- Done when: dotnet build succeeds with no errors and no bin/ or obj/ folders appear in git status.

1.2 Docker Compose infrastructure (done)

- Build: docker-compose.yml with PostgreSQL, Redis and RabbitMQ (management UI), healthchecks, named volumes, a named network, and credentials read from .env. Create .env.example with placeholders.
- Spec: section 18.
- Done when: docker compose ps shows all three healthy, select 1 works in PostgreSQL, Redis answers PONG, and the RabbitMQ page opens on port 15672.

1.3 TradeService starter (running; commit pending)

- Build: make TradeService a runnable ASP.NET Core API with Swagger, a /health endpoint, a temporary /api/v1/ping endpoint, and a service name read from configuration (overridable by an environment variable).
- Spec: sections 10, 20 (health), FR-22.
- Done when: Swagger opens, /health returns Healthy, and setting Service__Name changes the name in the log and the ping response. Then clear the variable and commit.

1.4 Shared error and pagination types

- Build: in TradeFolio.Common, a standard ProblemDetails-style error response (type, title, status, traceId, errors) and a reusable paged-result type (items, page, pageSize, total). Add a unit test project for Common and test both types.
- Spec: section 10 (standard error response), FR-19, FR-20.
- Done when: unit tests pass and an invalid request in Swagger returns the spec's error shape.

1.5 Docs and tracking files

- Build: docs/progress.md, docs/requirements-checklist.md (FR-01 to FR-22 with status, code location and test name columns), and docs/adr/0001-service-data-ownership.md comparing options for how Trade and Portfolio share the transaction (shared schema with one writer, or holdings and lots inside Trade Service with Portfolio as a read model).
- Spec: sections 4, 7, 26.
- Done when: the ADR records a decision and you can explain it in your own words. **[YOU]** make the final call.

Phase 2: FIFO accounting (pure code, no database)

2.1 Domain types

- Build: in TradeService/Domain, a Lot type (id, original and remaining quantity, price, purchased time) and a result type returning cost basis, proceeds, realized P&L and the updated lots. All decimal. No database or HTTP code.

- Spec: sections 6, 8.
- Done when: the project builds and the Domain folder has no reference to EF Core or HTTP.

2.2 FIFO tests first

- Build: a unit test project for TradeService with failing tests: the spec example (BUY 10 at 100, BUY 5 at 120, SELL 12 at 150 gives cost basis 1240, proceeds 1800, realized P&L 560, remaining 3 at 120), selling exactly one lot, selling everything, selling more than owned (must throw), equal timestamps (tie-break by id), and partial consumption of one lot.
- Spec: section 8, FR-08.
- Done when: tests exist and fail for the right reason. **[YOU]** add one edge case of your own.

2.3 FIFO implementation

- Build: the pure FIFO sell function so every test passes, without editing the tests.
- Spec: section 8.
- Done when: all FIFO tests pass and the test file is unchanged from 2.2.

2.4 Complexity note

- Build: docs/fifo.md explaining how the algorithm works, its time complexity, and why FIFO was chosen.
- Spec: sections 8, 28.
- Done when: you can explain it out loud without notes.

Phase 3: Database and BUY/SELL (week 2)

3.1 EF Core and first migration

- Build: EF Core with Npgsql in TradeService. First migration for Instruments, Trades, Holdings and HoldingLots, using numeric for money and quantity. user_id is a plain Guid column for now; the Users table arrives in Phase 5. The connection string comes from environment variables.
- Spec: sections 6, 9.
- Done when: the migration applies and the tables are visible in DBeaver or pgAdmin.

3.2 Database constraints and indexes

- Build: CHECK quantity > 0 and price > 0, CHECK remaining_quantity >= 0, unique (user_id, instrument_id) on Holdings, unique (user_id, idempotency_key) on Trades, plus the indexes listed in the spec.
- Spec: sections 6 (invariants), 9.
- Done when: inserting a negative quantity directly in SQL is rejected by the database.

3.3 Instrument seed data

- Build: about 10 seeded instruments (symbol, name, sector, currency) and GET endpoints to list them.
- Spec: sections 6, 22.
- Done when: instruments appear in the table and in Swagger.

3.4 Validation and error responses

- Build: FluentValidation for the trade request (symbol must exist, quantity and price above zero, side BUY or SELL), returning the standard error response.
- Spec: FR-05, FR-20.
- Done when: a negative quantity and an unknown symbol each return a clear 400.

3.5 BUY flow

- Build: the BUY side of POST /api/v1/trades in one transaction: upsert the holding (handles the first-BUY race), create a lot, update the holding quantity, insert the trade. The user id comes from a temporary header.
- Spec: section 7, 8, FR-04, FR-07.
- Done when: after a BUY the database shows one trade, one holding and one lot with matching quantities.

3.6 SELL flow

- Build: SELL using the exact algorithm in section 7: lock the holding row, re-read quantity after the lock, reject if too large, consume lots in FIFO order using the Phase 2 code, update lots and holding, insert the trade. Read Committed plus explicit row locking.
- Spec: section 7, FR-06, FR-07, FR-08.
- Done when: BUY 10, SELL 4 works and SELL 8 is rejected. **[YOU]** read the locking code and confirm the lock is taken before the quantity is read.

3.7 Idempotency-Key

- Build: same key and same body returns the original result; same key with a different body returns a conflict; two simultaneous requests with the same key create one trade. Keys are scoped per user.
- Spec: sections 7, 10.
- Done when: sending the same request twice creates one trade row.

3.8 Read endpoints

- Build: GET /api/v1/holdings, GET /api/v1/trades (paginated, ordered by executed_at descending then id descending, with a maximum page size) and GET /api/v1/trades/{id}.
- Spec: section 10, FR-04, FR-19.
- Done when: with 25 trades, page 1 and page 2 have no duplicates or gaps.

3.9 EXPLAIN ANALYZE document

- Build: pick two queries (trade history by user, FIFO lot lookup), run EXPLAIN ANALYZE with a realistic amount of data (for example 100,000 trades) and write docs/database/query-analysis.md explaining the plans and indexes.
- Spec: section 9, section 26 item 43.
- Done when: the document shows index scans, not full table scans. **[YOU]** run the queries yourself once.

Phase 4: Concurrency proof (week 2, the centerpiece)

4.1 Test fixture

- Build: TradeService.IntegrationTests with a shared Testcontainers PostgreSQL fixture that starts a real database, applies migrations and gives each test a clean state, using random ports.
- Spec: section 17.
- Done when: a simple test inserts and reads a row in the real container.

4.2 Core trade tests

- Build: integration tests through the real transaction path: BUY creates trade, holding and lot; partial SELL consumes FIFO lots correctly; oversell is rejected.
- Spec: section 17 (acceptance tests 15 to 17).
- Done when: all pass with no database mocks.

4.3 Concurrency test [YOU]

- Build: holding of 10 shares, two SELL 7 requests at the same time behind a synchronization barrier. Assert exactly one succeeds, one fails with insufficient quantity, final holding is 3, no negative quantity, one trade and one outbox event for the successful sell.
- Spec: section 7, FR-06.
- Done when: it passes, and it fails when you temporarily remove the row lock (proving the test is real). Restore the lock afterwards.

4.4 Repeat runs and extra races

- Build: run the concurrency test in a loop of 50 iterations. Add tests for BUY and SELL at the same time, two first-BUYs on a new instrument, and two requests with the same Idempotency-Key.
- Spec: section 7, section 29.
- Done when: all 50 iterations pass every time.

4.5 Isolation decision record

- Build: docs/adr/0002-isolation-and-locking.md explaining why Read Committed plus row locking, what problems it prevents, and when Serializable might be better.

- Spec: section 7, section 28.
- Done when: you can answer "Why Read Committed plus row locking?" without notes.

Phase 5: Authentication and security (week 6 items pulled forward)

5.1 Registration

- Build: Users table and POST /api/v1/auth/register. Validate email and password, store only a hashed password (ASP.NET Core Identity's hasher), reject duplicate emails ignoring case.
- Spec: FR-01, section 16.
- Done when: the password column holds a hash, never plain text.

5.2 Login and JWT

- Build: POST /api/v1/auth/login returning a short-lived JWT and GET /api/v1/users/me. Validate signature, expiry, issuer and audience. Signing key from environment variables. Same error for wrong email and wrong password.
- Spec: FR-02, section 16.
- Done when: no token gives 401, and an expired or tampered token is rejected.

5.3 Ownership and isolation

- Build: replace the temporary user-id header with the id from the JWT claim. Every trade, holding, analytics and notification query filters by that user. Return 404 for another user's resource.
- Spec: FR-03, section 16.
- Done when: **[YOU]** with two registered users, User A cannot see User B's trade by id.

5.4 Isolation tests

- Build: API integration tests proving User A cannot read, list or trade against User B's data, and that requests without a token fail.
- Spec: section 17 (acceptance test 14).
- Done when: tests pass.

5.5 Gateway and versioning

- Build: a Gateway project routing /api/v1/* to the services, validating JWTs and returning correlation ids. Services are not meant to be called directly.
- Spec: sections 3, 4, FR-22.
- Done when: the API works through the gateway port.

5.6 Rate limiting and CORS

- Build: token-bucket rate limiting (stricter on login) and an explicit CORS policy for the frontend origin.

- Spec: FR-21, section 16.
- Done when: sending many requests quickly returns 429.

5.7 Security review

- Build: review the code against section 16: secrets in source control, passwords or tokens in logs, error messages that leak details, missing validation. Write down findings, then fix them.
- Spec: section 16.
- Done when: the findings list is empty or each item is fixed.

Phase 6: Outbox and RabbitMQ (week 3)

6.1 Event contract

- Build: the TradeCompleted event in TradeFolio.Contracts, exactly as in the spec, including schemaVersion and correlationId.
- Spec: section 11, FR-09.
- Done when: it builds and matches the spec's JSON.

6.2 Outbox in the same transaction

- Build: the OutboxMessages table and an outbox row written in the same transaction as the trade, with a unique event_id.
- Spec: sections 7 (step 10), 11.
- Done when: after a trade there is exactly one outbox row with published_at empty.

6.3 RabbitMQ topology

- Build: exchange, notification queue and dead-letter queue defined in infrastructure/docker/rabbitmq/definitions.json and loaded by Docker Compose.
- Spec: section 11, FR-12.
- Done when: the RabbitMQ page shows the exchange and queues.

6.4 Outbox publisher

- Build: a background service that reads unpublished rows (safe for several instances with FOR UPDATE SKIP LOCKED), publishes with publisher confirms, marks rows published and retries on failure.
- Spec: section 11, FR-09.
- Done when: a trade produces a message in RabbitMQ and the row gets a published_at.

6.5 Broker outage test

- Build: an integration test that stops RabbitMQ, completes a trade (must still succeed), confirms the outbox row stays pending, restarts RabbitMQ and confirms eventual publication.

- Spec: section 11, FR-09, section 17 (acceptance test 22).
- Done when: it passes and you can show it live in the demo.

Phase 7: Notification service (week 3)

7.1 Service skeleton and consumer

- Build: NotificationService as its own runnable project with its own Dockerfile and health check, consuming TradeCompleted and creating a notification.
- Spec: FR-10, section 4.
- Done when: a trade creates a notification row.

7.2 Idempotency

- Build: NotificationDelivery table with a unique event_id. Already-processed events are acknowledged without sending again. Two consumers receiving the same event are handled by the unique constraint. Acknowledge only after the record is saved.
- Spec: FR-11, section 11.
- Done when: replaying the same message twice yields one notification.

7.3 Provider interface and mock

- Build: notification sending behind an interface, plus a mock provider for local development.
- Spec: section 11.
- Done when: the mock logs the notification and no real email service is needed.

7.4 Retry and dead-letter queue

- Build: exponential backoff with jitter for temporary failures, recorded attempts and last error, exhausted messages routed to the dead-letter queue. Malformed or unknown-schema messages go straight to the dead-letter queue.
- Spec: FR-12, section 11.
- Done when: making the mock fail shows retries and then the dead-letter queue in the RabbitMQ page.

7.5 Notification tests

- Build: integration tests with a real RabbitMQ container: duplicate event gives one notification, transient failure retries and succeeds, exhausted failure ends in the dead-letter queue.
- Spec: section 17 (acceptance tests 20 and 21).
- Done when: all pass.

7.6 Notifications endpoint

- Build: GET /api/v1/notifications (paginated, user-scoped) through the gateway.

- Spec: section 10, FR-03, FR-19.
- Done when: User A sees only their own notifications.

Phase 8: Market data and Redis (week 3)

8.1 Provider adapter

- Build: MarketDataService with an IMarketDataProvider interface and one adapter for a free price provider. Check the provider's limits and terms first and record the choice. Keep the rest vendor-neutral.
- Spec: section 12, FR-13.
- Done when: you can fetch one real price for AAPL.

8.2 Redis cache

- Build: caching with a TTL; hits skip the provider; misses call the provider and store the result only after success; protect against many requests hitting the provider when a hot key expires.
- Spec: FR-14, section 12.
- Done when: the second request for the same price comes from Redis.

8.3 Failure behavior

- Build: Redis down means call the provider directly with a timeout; provider down means return a value clearly marked stale (within a maximum age) or a clear error; reject invalid prices (zero or negative). Trades never fail because of market data.
- Spec: section 12, section 29.
- Done when: stopping Redis, then the provider, one at a time, still lets trades work.

8.4 Price endpoint

- Build: GET /api/v1/market-prices/{symbol} returning price, observed time, source and a stale flag.
- Spec: section 10, FR-13.
- Done when: the stale flag is correct.

8.5 Market data tests

- Build: tests for cache hit avoiding the provider call, miss populating Redis, Redis failure fallback, and provider timeout returning stale or an error.
- Spec: section 17 (acceptance test 23).
- Done when: all pass.

Phase 9: Analytics (week 4)

For every analytics task, **[YOU]** calculate the expected numbers by hand or in a spreadsheet before the tests are written.

9.1 Portfolio value and unrealized P&L

- Build: pure calculators for portfolio value (quantity times latest price) and unrealized P&L (market value of open lots minus remaining cost basis). Define what happens when a holding has no price. An empty portfolio must not divide by zero.
- Spec: section 13, FR-15.
- Done when: unit tests match your hand-calculated numbers.

9.2 Realized P&L

- Build: realized P&L from FIFO cost basis consumed by sells, including fully closed positions.
- Spec: sections 13, 8.
- Done when: the spec example gives 560.

9.3 Allocation and sector exposure

- Build: allocation per instrument and exposure per sector as percentages of total value, handling rounding and the empty portfolio.
- Spec: section 13.
- Done when: percentages add up sensibly and an empty portfolio returns an empty result.

9.4 Trade frequency

- Build: sliding-window trade counts (or SQL window functions), a documented rule for inclusive window boundaries, and the time complexity.
- Spec: sections 8, 13.
- Done when: tests cover a trade exactly on the boundary and windows with zero trades.

9.5 Best and worst performers

- Build: rank instruments by a clearly defined performance measure with a deterministic tie-break.
- Spec: section 13.
- Done when: ties always come out in the same order.

9.6 Insights

- Build: deterministic insights (concentration, activity, performance) with fixed rules and thresholds that stay sensible on tiny portfolios.
- Spec: FR-16.
- Done when: a one-trade portfolio produces no alarming or odd messages.

9.7 Analytics endpoints and tests

- Build: AnalyticsService with endpoints summary, pnl, allocation, sectors, frequency and performers, and tests comparing results with your independently calculated values.
- Spec: section 10, section 17 (acceptance test 24).
- Done when: the API numbers match your hand calculation exactly.

Phase 10: React dashboard (week 4)

10.1 Frontend scaffold

- Build: src/Frontend/tradefolio-web with React, TypeScript and Vite, a typed API client, folders for auth, features, charts and components, and a Dockerfile.
- Spec: sections 14, 24.
- Done when: npm run dev shows a page.

10.2 Login and registration

- Build: login and register pages, auth state, token handling, protected routes and expired-token handling.
- Spec: section 14, FR-17.
- Done when: you can log in, refresh and log out in the browser.

10.3 Holdings and trade history

- Build: a holdings table and a paginated trade history table with loading, empty and error states.
- Spec: section 14, FR-17.
- Done when: the numbers match the database.

10.4 Buy/Sell form

- Build: a form with client-side validation, server error display and double-click protection using an Idempotency-Key.
- Spec: section 14, FR-05.
- Done when: double-clicking Submit creates one trade.

10.5 Analytics dashboard

- Build: KPI cards, allocation chart, sector chart, trade-frequency chart, performers and insights in the layout from section 14, using Recharts or ECharts, with data transformation kept separate from display components.
- Spec: section 14, FR-17.
- Done when: every section shows real data and the empty state looks fine for a new user.

10.6 Notifications and price freshness

- Build: a recent notifications panel and a price freshness indicator (live, cached, stale).

- Spec: section 14.
- Done when: making a trade makes a notification appear.

10.7 Frontend tests

- Build: tests for the login form, the trade form and dashboard rendering with mocked API data.
- Spec: section 17 (frontend).
- Done when: tests pass.

Phase 11: BI dashboard (week 6)

11.1 Analytics views

- Build: SQL views in bi/sql for portfolio KPIs, P&L trend, sector exposure, allocation, trade activity and performers, based on the same data the app uses.
- Spec: section 15, FR-18.
- Done when: each view returns data in DBeaver that matches the app.

11.2 Build the dashboard [YOU]

- Build: connect Power BI or Tableau (pick one) to the views and build the pages.
- Spec: section 15.
- Done when: the dashboard shows KPIs, P&L trend, sector exposure, allocation, trade activity and performers.

11.3 Cross-check

- Build: a short checklist or script comparing BI numbers with the API for one test user.
- Spec: section 15.
- Done when: the numbers match.

Phase 12: Production engineering and finish (weeks 5 and 6)

12.1 Docker hardening

- Build: multi-stage Dockerfiles for every .NET service with small runtime images, non-root users and health checks, and no secrets baked into images.
- Spec: section 18.
- Done when: images are small and containers run as non-root.

12.2 Full stack in one command

- Build: every service and the frontend in docker-compose.yml with health-based startup order, and migrations that run safely (not every container racing).
- Spec: section 18, acceptance test 27.

- Done when: after deleting your volumes, docker compose up --build starts everything from scratch.

12.3 GitHub Actions

- Build: a workflow that restores, builds, runs unit tests, runs integration tests (Testcontainers), builds Docker images and scans for secrets.
- Spec: section 19.
- Done when: a pull request shows a green check.

12.4 OpenTelemetry

- Build: tracing, metrics and structured logs (trade_id, event_id, correlation_id) across services, including trace context through RabbitMQ messages.
- Spec: section 20.
- Done when: one trace shows API, trade service, database and the notification consumer.

12.5 Prometheus and Grafana

- Build: both in the compose file, with a dashboard for request count, errors, latency, queue depth, retries and cache hits and misses.
- Spec: section 20.
- Done when: the dashboard shows data after a few trades.

12.6 Terraform

- Build: Terraform for the AWS path (ECR, ECS Fargate, RDS, CloudWatch) under infrastructure/terraform with dev and prod environments. Validate only; do not apply. Document cost and how to destroy.
- Spec: section 21.
- Done when: terraform validate passes.

12.7 Documentation

- Build: README with one-command setup, architecture diagram, API docs link, testing instructions, deployment notes and rollback steps, plus docs/demo-script.md from section 27.
- Spec: sections 26, 27, 31.
- Done when: **[YOU]** you can follow the README on a fresh clone without getting stuck.

12.8 Final audit

- Build: check the whole repo against section 26 (Definition of Done) and section 31 (Final Deliverables). Mark each item done, partly done or missing, with the file or test as proof.
- Spec: sections 26, 31.

- Done when: gaps are fixed and the full demo script from section 27 runs end to end.

Where each requirement is built

- FR-01 Registration: 5.1
- FR-02 Authentication: 5.2
- FR-03 User isolation: 5.3, 5.4, 7.6
- FR-04 Trades: 3.5, 3.6, 3.8
- FR-05 Validation: 3.4, 10.4
- FR-06 No overselling: 3.6, 4.3, 4.4
- FR-07 Atomicity: 3.5, 3.6
- FR-08 FIFO: 2.1 to 2.4, 3.6
- FR-09 Events: 6.1, 6.2, 6.4, 6.5
- FR-10 Notifications: 7.1
- FR-11 Idempotency: 7.2
- FR-12 Retry and DLQ: 6.3, 7.4
- FR-13 Market data: 8.1, 8.4
- FR-14 Redis: 8.2, 8.3
- FR-15 Analytics: 9.1 to 9.5, 9.7
- FR-16 Insights: 9.6
- FR-17 Dashboard: 10.1 to 10.7
- FR-18 BI: 11.1 to 11.3
- FR-19 Pagination: 1.4, 3.8
- FR-20 Errors: 1.4, 3.4
- FR-21 Rate limiting: 5.6
- FR-22 Versioning: 1.3, 5.5